Lesson 10 — What Each Notebook Shows

Notebook guide, lesson 10 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

18 December 2026 · three notebooks

Contents

Notebook 01 — `01_what_a_convolution_is` **1**
What a convolution is 1

Notebook 02 — `02_convnets_on_wafers` **2**
A convolutional network on the wafer line 2

Notebook 03 — `03_transfer_and_synthesis` **4**
Transfer learning, and what the course adds up to 4

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

Notebook 01 — `01_what_a_convolution_is`

What a convolution is

What it is for. Lesson 9 ended on a promise: every network in it treated the pixels as unrelated numbers. This notebook builds the operation that does not, and checks its two defining properties before any training happens.

The data. 4,000 wafer dies, 24×24 pixels, 50.8% defective. The grading station is wrong about **2%** of verdicts by design (2.55% realised in this batch, which the notebook shows is **2.5 standard deviations high — a draw, not a bug**).

The concepts, in order.

1. **Why the problem is different.** The defect is **local** — a handful of pixels out of 576 — and the background is **not flat**, so a detector must respond to local contrast rather than to brightness.
2. **What a dense layer would have to learn**, in parameters:

layer	parameters
dense, 256 units	147,712
conv, 8 kernels of 3×3	**80**
conv, 32 kernels of 5×5	832

Eighty against a hundred and forty-seven thousand — and the convolutional layer produces *more* numbers, not fewer. The saving is weight sharing, not size.

3. **The operation in eleven lines**, matched against scipy to $\mathbf{8.9 \times 10^{-1}}$.
4. **The output-size formula**, $\lfloor (n + 2p - f)/s \rfloor + 1$, checked against the measured shapes in six configurations — all `True`.
5. **Kernels chosen by hand.** A 3×3 array of nine numbers expressing "darker than its surroundings": strongest response **3.129** on a defective die against **1.118** on a clean one.
6. **The property the whole idea rests on.** A kernel whose weights **sum to zero** returns exactly zero on any constant patch, so it ignores smooth background variation. And **equivariance**, verified: convolve-then-shift equals shift-then-convolve to **0.00e+00** for every shift tried.
7. **Pooling: from "where" to "whether".** The strongest response survives every pooling step — **3.129, 3.129, 3.129, 3.129** — and what is discarded is the location, which for grading a die is exactly what we are happy to lose.

The two-minute version. *A defect is a handful of pixels out of 576, and the background is not flat, so what we need is a detector of local contrast that works anywhere on the die. A dense layer of 256 units needs 147,712 parameters and learns each position separately; eight 3×3 kernels need 80 and share them across every position. We implement convolution in eleven lines and match scipy to fifteen decimal places. Two properties make it work: a kernel summing to zero ignores flat brightness entirely, and convolution is equivariant to translation — shifting then convolving is exactly the same operation as convolving then shifting, to zero. Pooling then throws away where the response was and keeps whether it happened.*

Notebook 02 — `02_convnets_on_wafers`

A convolutional network on the wafer line

What it is for. The counting and the checking are worth nothing until they are measured against the alternative — lesson 9's dense network, on exactly the same images.

The setup. 8,000 training images, 2,000 test, ceiling realised on the test set **0.9800**.

The concepts, in order.

1. **Two architectures.** The dense network is lesson 9's, unchanged. The convolutional one alternates 3×3 kernels with pooling and ends in a **global** maximum — one number per kernel, saying *whether*, not *where*.
2. **Efficiency, in parameters per point of accuracy: dense 3,086, convolutional 16.**

3. **And a result worth reading twice.** The convolutional network scores 0.9800 against the *recorded* grade and **1.0000 against the true grade**. Its 0.98 is not a modelling shortfall at all — **it is the grading station's own 2% error rate**.

4. **A defect where none has been seen**, which is the experiment of the lesson. Defects confined to the top band in training, the bottom band in one test set:

network	defect in trained band	defect moved	cost of moving it
dense (lesson 9)	0.8567	**0.4617**	−0.395
convolutional	0.9800	0.9847	+0.005

   The dense network falls to chance. It had learned "dark pixels up here", which is not the same thing as "a defect".

5. **Can augmentation buy the same thing?** 3,000 images become 15,000, every copy randomly shifted: the dense network climbs **0.4617 → 0.5423**, a gain of 0.08 and still near chance. **Augmentation helps and does not come close.**

6. **How much data does each need?** The convolutional network reaches the ceiling at **500 images**; the dense one, at **8,000**, is still **0.235 short**.

7. **What is it actually using?** One fixed permutation applied to every image:

task	network	as photographed	pixels permuted
is there a defect?	dense	0.6985	0.7097
	convolutional	0.9798	0.9800
which defect is it?	dense	0.6615	0.6478
	convolutional	0.9970	**0.9523**

   The dense network does not notice — for it, permuting pixels is a relabelling of input units. The convolutional network loses nothing on the easy task and **4.5 points on the harder one**, which is where its assumption about neighbouring pixels was actually paying.

8. **What the first layer learned.** Kernel weight sums: seven of eight are below 0.6 in absolute value against a scale of 2.16 — **they discovered the sum-to-zero property nobody required of them**.

The two-minute version. *Same images, two networks. The convolutional one scores 0.9800 — and against the true grade it is right about every single die, so that 0.98 is the grading station's error rate, not the model's. Then the experiment: train with defects only in the top half of the die and test with them in the bottom. The dense network drops from 0.86 to 0.46, chance, because it learned "dark pixels up here"; the convolutional one goes from 0.98 to 0.98. Augmentation with five times the data recovers eight points of the forty lost. And the convolutional network reaches the ceiling with 500 images where the dense one, with 8,000, is still 23 points short.*

Notebook 03 — `03_transfer_and_synthesis`

Transfer learning, and what the course adds up to

What it is for. A new defect appears on Monday; by Friday there are forty labelled photographs. The previous notebook needed hundreds. This one asks what can be borrowed — and then puts all ten lessons on one table.

The concepts, in order.

1. **Pre-training on a task nobody wants solved.** Naming which defect (clean, scratch, particle) rather than grading pass/fail, because it teaches more than one detector: **0.9993** — and the notebook says plainly that **this number is not a result, it is a receipt** showing the source task was learned.

2. **Starting from those weights instead of from noise:**

labelled images	from scratch	warm-started	gain
25	0.5100	0.5100	0.000
50	0.5707	0.6603	+0.090
100	0.7220	**0.8640**	**+0.142**
200	0.7270	0.8940	+0.167
400	0.8447	0.7877	**−0.057**

   Read the gain column *across* the rows: nothing at 25, most at 100–200, and **negative at 400**, where the borrowed weights have become a constraint rather than a head start.

3. **When transfer makes things worse.** A badly chosen source — kernels trained only on dark scratches — is **stuck at chance** (0.5047 at 100 images) because it is committed to reporting darkness where the new defect is faint.

4. **What the course adds up to**, every family on the raw 576 pixels and then on four hand-made features:

family	raw pixels	4 hand-made features
logistic regression (L3, L4)	0.8905	0.9800
k-nearest neighbours (L6)	**0.4970**	0.9800
support vector machine (L6)	0.9660	0.9800
random forest (L7)	0.9780	0.9795
dense network (L9)	0.7430	— (213,761 parameters)
convolutional network (L10)	**0.9800**	— (1,537 parameters)

   Four well-chosen numbers put every classical method on the ceiling, and the convolutional network gets there from the raw pixels with 1,537 parameters. That contrast is the closing argument of the course: representation is not a detail that precedes the model, it *is* most of the model.

The two-minute version. *Forty images of a new defect is not enough to train anything, so we borrow. Pre-train on a related task — naming which defect rather than grading pass/fail — and then*

warm-start. The gain is zero at 25 images, fourteen points at 100, and negative at 400, where the borrowed weights start to constrain rather than help. A badly chosen source, trained only on dark scratches, stays at chance on a faint defect. Then the synthesis: run every family from the whole course on the raw pixels and on four hand-made features. On raw pixels k-NN is at chance and logistic regression at 0.89; on four good numbers every classical method sits exactly on the ceiling. The convolutional network gets to the same place from the raw pixels with 1,537 parameters — which is what the whole course has been about.